Healthcare Patient Management Platform - GP 3

Class: ENPM818T

Group: Group 6

Term: Spring 2026

Professor: Zeid Kootbally

Group Members

Name	UID
Pushkar Vishwas	122141698
Etsubdink Workalemahu Yergashewa	118523194
Louis Tafah	119478042
Adyasha Mishra	122011410

Executive Summary

This project presents the design and implementation of a secure healthcare management platform built using a polyglot database architecture. The system integrates three specialized databases: PostgreSQL, MongoDB, and Neo4j to support clinical operations, patient record management, prescription safety analysis, and medical knowledge graph processing. The primary objective of the project is to demonstrate how different database technologies can be combined systematically to improve scalability, flexibility, and clinical intelligence within a healthcare environment.

The architecture separates structured transactional healthcare data, semi-structured clinical documentation, and highly connected drug-interaction relationships into databases optimized for each workload. PostgreSQL manages highly consistent transactional data such as patients, appointments, billing, providers, prescriptions, and admissions. MongoDB stores flexible clinical records, including physician notes, symptom logs, and medical histories that may vary significantly between patients. Neo4j powers the clinical decision support engine by modeling relationships between drugs, allergies, diagnoses, and contraindications using graph-based traversal queries.

A key achievement of this system is the implementation of a drug safety and prescription validation workflow. Before a prescription is finalized, the application automatically checks for known drug-to-drug interactions, allergy conflicts, and contraindications through the Neo4j knowledge graph. This reduces clinical risk and demonstrates how graph databases can enhance patient safety in modern healthcare systems. Additional accomplishments include secure database integration, normalized relational schema design, cross-database referencing strategies, and support for operational, financial, and clinical analytics within the healthcare system.

Data Partitioning Rationale

The system uses a polyglot persistence strategy, where each database technology is selected based on the characteristics of the data and workload requirements. This design improves overall system efficiency by allowing each database engine to perform tasks it is best optimized for.

PostgreSQL was selected and used because of its ability to process highly structured and transactional data. It was also selected because it provides a strong Atomicity, Consistency, Isolation, and Durability (ACID) compliance. This property makes it suitable for guaranteeing data integrity, even in the face of errors, system crashes, or concurrency. It was also selected due to its row-level security, audit logging capabilities, support for complex SQL queries, and advanced indexing. HIPAA compliance also requires strict access controls, encryption of data at rest and in transit, and comprehensive audit trails, all of which PostgreSQL provides. The data stored or processed within this database technology includes patients, providers, appointments, prescriptions, billing records, insurance plans, admissions, laboratory orders, and results.

MongoDB was selected and used for storing semi-structured and rapidly changing healthcare data, such as clinical notes and patient histories. These notes often vary between providers and cannot always fit into the rigid relational database schemas. MongoDB also allows scalability and schema flexibility, and some of the data processed within this database technology includes physician notes, diagnostic observations, systems tracking notes, progress logs, and other forms of unstructured medical records.

Neo4j is used for graph-based relationships, particularly for drug interactions and clinical decision support. The healthcare data often involves complex relationships, drug-to-drug, patient-to-condition, and provider-to-patient. Graph databases enable efficient traversal of these

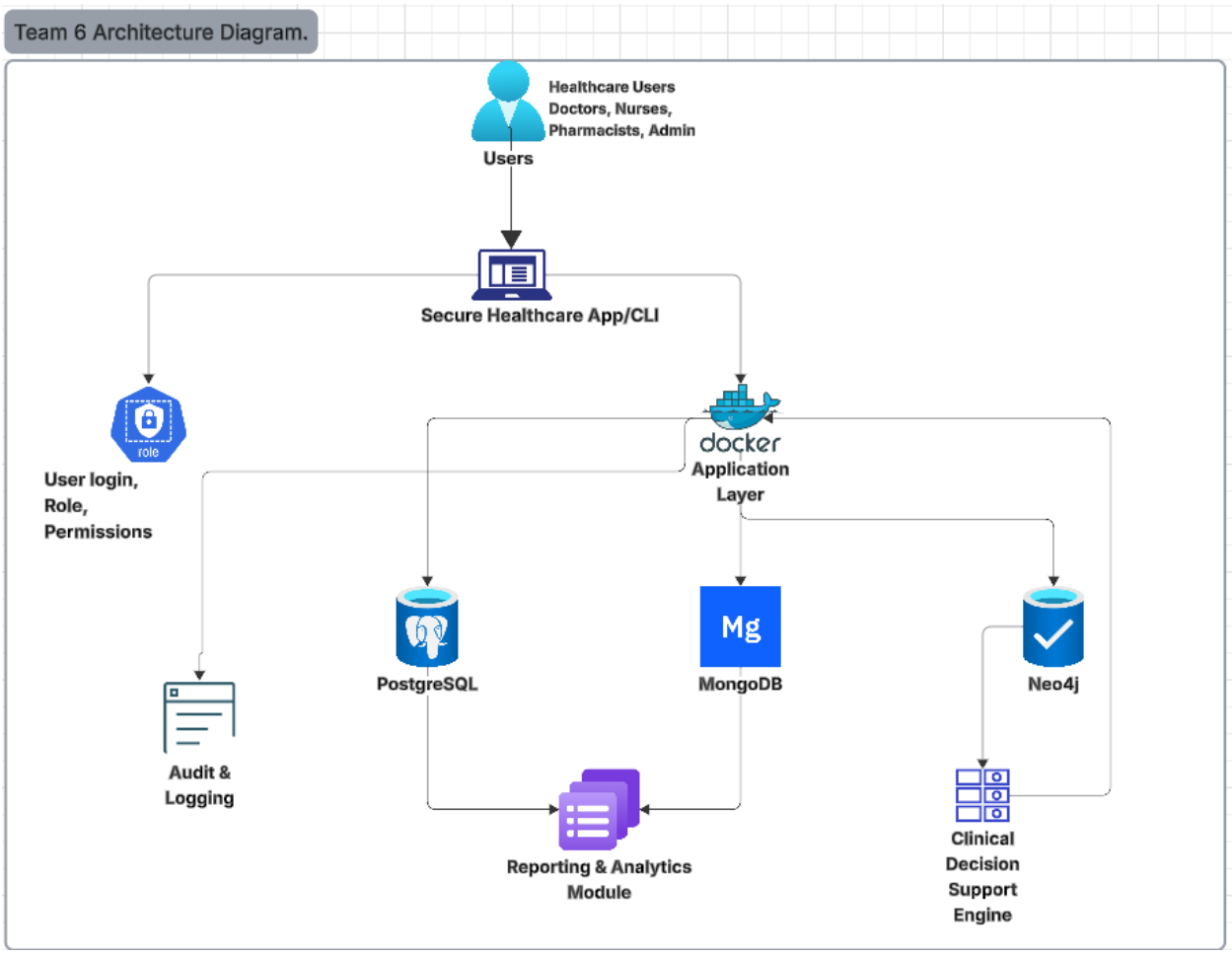
relationships, allowing real-time safety checks that would be inefficient in relational systems. The Neo4j database technology stores data that includes the following contraindications, allergy associations, medication categories, drug interaction relationships, and therapeutic relationships.

Architecture Overview

The architecture overview focuses on three major areas: the system architecture diagram, the data flow diagrams for clinical workflows, and the component descriptions for each database layer. Together, these elements show how the healthcare management platform is organized, how data moves through the system, and how PostgreSQL, MongoDB, and Neo4j each support a specific part of the overall healthcare environment.

System Architecture Diagram

The system architecture is designed as a polyglot healthcare platform that separates user interaction, business logic, and database services into different layers. At the top level, healthcare users such as physicians, nurses, pharmacists, billing staff, and administrators access the system through a command-line interface.

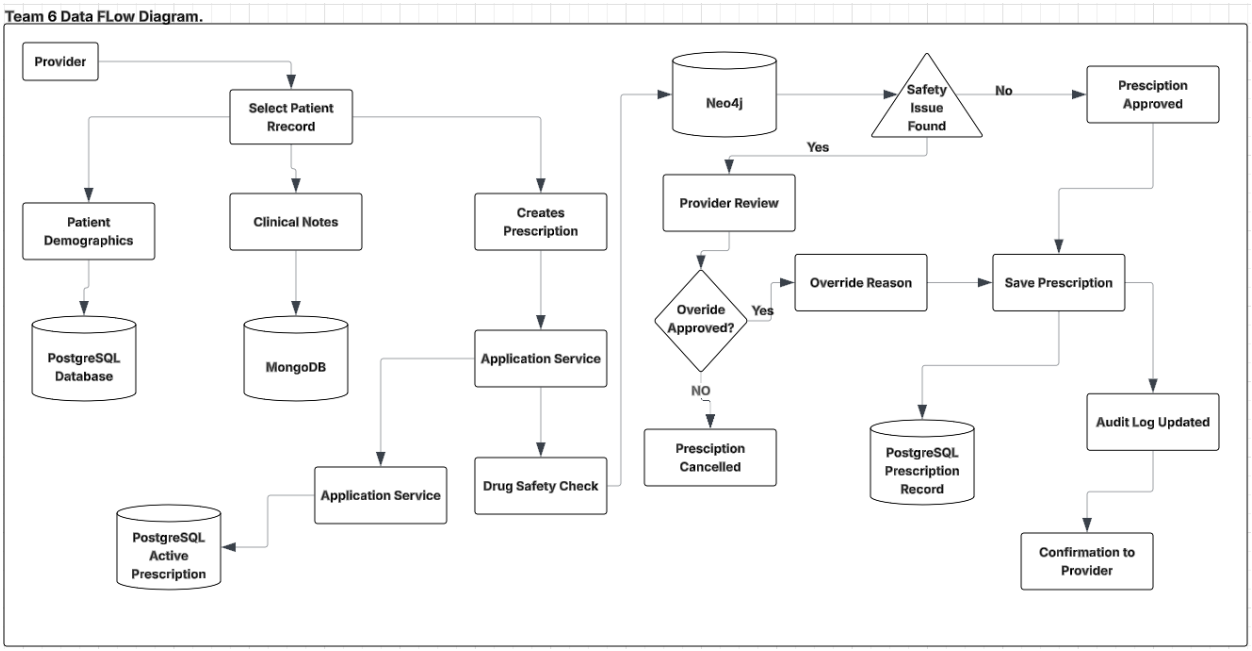


User requests are sent to the application/service layer, which handles authentication, authorization, validation, business rules, API processing, and audit logging. This layer acts as the central controller between the user interface and the three database systems. It determines which database should process each request based on the type of data being handled. The database layer contains three specialized databases:

- PostgreSQL for structured transactional records
- MongoDB for semi-structured clinical documentation
- Neo4j for graph-based drug interaction and clinical decision support

Data Flow Diagrams for Clinical Workflows

The data flow diagrams explain how information moves through the healthcare platform during clinical operations. The most important workflows include patient registration, appointment scheduling, clinical documentation, and prescription safety validation.



In the patient registration workflow, administrative staff enter patient demographic and insurance information into the application interface. The application validates the required fields, checks for duplicate patient records, and stores the structured patient data in PostgreSQL. For the appointment scheduling workflow, a staff member or provider creates appointments for patients. The application then checks the provider's availability, appointment date, appointment time, and patient information. PostgreSQL serves as an enforcer for scheduling constraints to prevent duplicate appointments or provider double-booking. During the clinical documentation workflow, providers enter physician notes, symptoms, observations, visit summaries, and progress notes. Since clinical notes can vary in structure, the application stores these records in

MongoDB. The prescription safety workflow is the most important clinical decision support process. When a provider attempts to prescribe medication, the application first retrieves the patient's existing medications, allergies, diagnoses, and relevant clinical information. The prescription request is then checked against the Neo4j knowledge graph for drug-to-drug interactions for allergy conflicts, contraindications, or medication class risks. If a safety issue is detected, the system generates an alert before the prescription is finalized. If no major issue is found, the prescription is approved and saved in PostgreSQL.

Component Descriptions for Each Database Layer

PostgreSQL Database Layer

PostgreSQL serves as the system's primary transactional database. It stores structured healthcare data that requires accuracy, consistency, and strong relationships between entities. This includes patients, providers, appointments, prescriptions, medications, insurance plans, billing records, admissions, lab orders, and lab results.

PostgreSQL supports the platform by enforcing primary keys, foreign keys, unique constraints, and normalized table relationships. It is responsible for maintaining the integrity of operational healthcare data. It also supports reporting queries, appointment scheduling, billing calculations, and prescription record keeping.

MongoDB Database Layer

MongoDB stores semi-structured and flexible clinical data. This includes physician notes, symptom histories, treatment observations, progress logs, diagnostic notes, and visit summaries. These records may vary from patient to patient and from provider to provider, making MongoDB more appropriate than a rigid relational database schema.

MongoDB improves system flexibility because new fields can be added to clinical documents without requiring a major schema redesign. It also supports fast retrieval of patient-specific clinical documentation and allows the system to store detailed medical narratives in a format close to how providers write them.

Neo4j Database Layer

Neo4j serves as the clinical knowledge graph and decision support database. It stores connected healthcare relationships involving medications, contraindications, drug classes, and therapeutic categories. In this implementation, Neo4j supports real-time prescription safety checks by evaluating relationships between a proposed medication and the patient's currently active medications retrieved from PostgreSQL. This allows the system to quickly identify drug-to-drug interactions and contraindication risks before a prescription is inserted into the transactional healthcare database.

Database Design Decision

PostgreSQL Design Decisions

PostgreSQL serves as the core transactional database within the healthcare management system. It was selected primarily because of its strong ACID compliance, reliability, and support for highly normalized relational schemas. Healthcare systems require strict consistency because incorrect or incomplete data can directly affect patient safety, which could lead to financial loss and public distrust.

The relational data schema was designed using normalization principles up to Third Normal Form (3NF) to minimize redundancy and eliminate update anomalies. Some of the major entities used in the project include:

- PATIENT

- EMERGENCY_CONTACT
- ALLERGY
- HOSPITAL
- DEPARTMENT
- PROVIDER
- APPOINTMENT
- PRESCRIPTION
- MEDICATION
- LAB_ORDER
- LAB_ORDER_ITEM
- LAB_TEST
- LAB_RESULT
- ADMISSION
- INSURANCE_PLAN
- PATIENT_INSURANCE
- BILLING_RECORD
- CLAIM

Primary and foreign key constraints enforce referential integrity between tables, while additional constraints and indexes improve data validation and query performance. For example, unique appointment scheduling constraints are used to prevent double-booking providers. DEA validation rules are used to support controlled substance prescription requirements, and audit timestamps are used to improve accountability and compliance monitoring (HIPAA) requirements. PostgreSQL was also selected because of its support for complex joins and

reporting queries, stored procedures and triggers, transaction rollback capabilities, row-level security, encryption support, and backup and recovery mechanisms. These capabilities make PostgreSQL highly suitable for managing mission-critical healthcare operations.

MongoDB Design Decisions

MongoDB was selected to handle semi-structured healthcare information that changes frequently or varies significantly between patients and providers. Healthcare documentation is often difficult to model in rigid relational schemas because different specialties and providers record information differently. MongoDB's document-oriented model allows flexible JSON-like structures that can evolve without requiring expensive schema migrations. This flexibility improves adaptability and accelerates development. Data collections stored within MongoDB include:

- Clinical notes
- Physician observations
- Symptom tracking records
- Patient progress logs
- Medical histories
- Visit summaries
- Diagnostic narratives

MongoDB supports horizontal scalability and high-volume document storage, making it appropriate for large healthcare environments generating extensive clinical documentation. It was the team's NoSQL database technology of choice due to its flexible schema design, high writing performance, scalable distributed architecture, efficient document retrieval, and support

for native JSON formats. MongoDB complements PostgreSQL by managing data that does not require rigid relational enforcement but still provides high operational value.

Neo4j Design Decisions

Neo4j was selected to power the clinical decision support and medication safety engine. Healthcare environments contain highly interconnected relationships between medications, diseases, allergies, symptoms, and contraindications. These relationships are difficult and inefficient to process using traditional relational joins. Graph databases excel at relationship traversal and connected-data analysis. Neo4j stores healthcare knowledge as nodes and relationships, enabling rapid safety analysis during prescription processing. A few examples of graph relationships include:

- DRUG_INTERACTS_WITH
- DRUG_CONTRAINDICATED_FOR
- PATIENT_HAS_ALLERGY
- MEDICATION_TREATS_CONDITION
- CONDITION_ASSOCIATED_WITH_SYMPTOM

When providers prescribe medication, Neo4j traverses these relationships in real time to identify safety risks before prescriptions are finalized. Neo4j was selected because it provides fast relationship traversal, efficient connected-data modeling, real-time graph analytics, flexible schema evolution, and enhanced decision support capabilities. These design capabilities significantly improve patient safety and further show how graph databases can enhance modern healthcare systems.

Clinical Decision Support Design

The clinical decision support capability implemented in this healthcare management platform focuses on prescription safety validation. The purpose of this feature is to reduce medication-related clinical risk by evaluating a newly proposed prescription against the patient's currently active medications before the prescription is committed to the transactional database.

This feature was implemented as part of the GP3 cross-database integration requirement and demonstrates coordinated use of PostgreSQL, Neo4j, and the Python application service layer.

The implemented workflow supports two clinical outcomes:

- detection and blocking of unsafe prescriptions
- approval and insertion of safe prescriptions into the healthcare system

This functionality simulates a real-world clinical medication safety check workflow commonly used in electronic health record systems.

Workflow Design

The workflow begins when a provider selects Prescription Safety Check from the GP3 cross-database CLI menu and enters:

- patient ID
- proposed medication name

The system then performs the following steps:

1. Validates the patient in PostgreSQL.
2. Retrieves active prescriptions from PostgreSQL.
3. Extracts medication names from the patient's current prescriptions.
4. Send the medication list and propose medication to Neo4j.
5. Queries the knowledge graph for medication interaction relationships.

6. If an interaction exists, it blocks prescription creation.
7. If no interaction exists, provider approval allows and inserts the prescription into PostgreSQL.

This ensures medication safety analysis occurs before the prescription becomes part of the official healthcare record.

Technical Component Responsibilities

Component	Responsibility
PostgreSQL	Stores patient data, active prescriptions, and approved prescription inserts
Neo4j	Evaluates medication interaction relationships
Python Service Layer	Orchestrates validation and decision logic
CLI Interface	Provider interaction and result display

Validation Testing

Unsafe Prescription Scenario

Test case:

- Patient ID: 35
- Proposed Medication: Clonazepam

The patient had an active prescription for Oxycodone.

The Neo4j knowledge graph detected a contraindicated drug interaction between the proposed medication and the patient's active prescription.

Oxycodone + Clonazepam → Risk of respiratory depression

System result:

- prescription blocked
- warning displayed
- no PostgreSQL insertion performed

```
C:\Users\louis\school-projects\Healthcare-Management\GP2_Healthcare_team_6>docker exec -it healthcare_app python -m src.cli.main


##################################################
#                                                #
#       HEALTHCARE MANAGEMENT SYSTEM             #
#                                                #
##################################################

1. Patient Management
2. Appointment Management
3. Prescription Management
4. Quick View - Upcoming Appointments
5. Quick View - Active Prescriptions
6. Quick View - Polypharmacy Report
7. GP3 Cross-Database Operations
0. Exit

Enter your choice: 7

======================================
      GP3 CROSS-DATABASE OPERATIONS
======================================
1. Complete Patient Record
2. Prescription Safety Check
0. Back to Main Menu

Enter choice:
```

Safe Prescription Scenario

Test case:

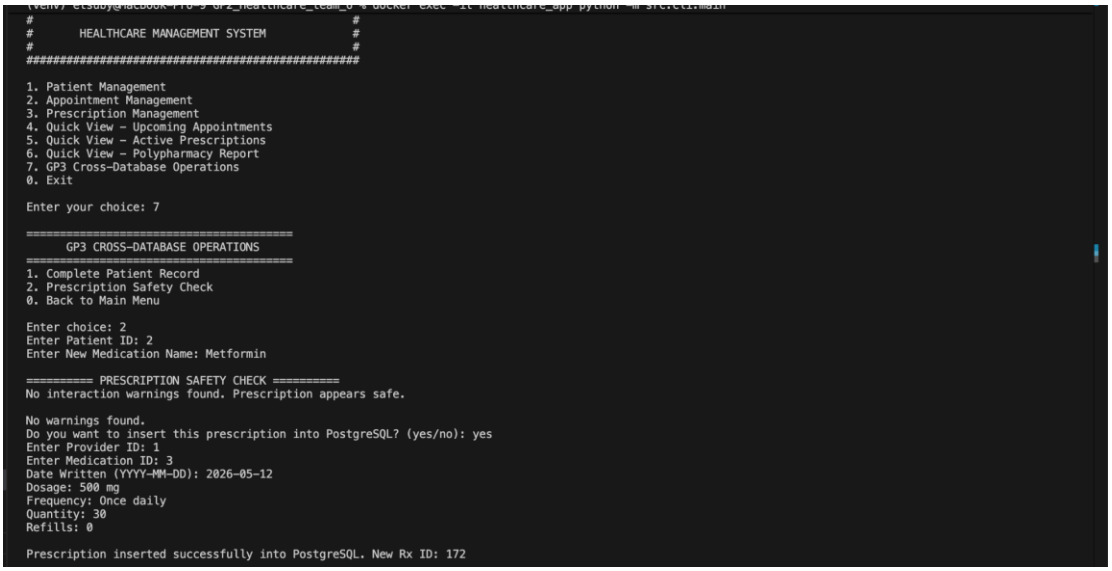
- Patient ID: 2
- Proposed Medication: Metformin

No medication interactions were detected.

The provider approved the prescription, and the system successfully inserted the prescription into PostgreSQL:

New Rx ID: 171

This confirms that the decision support workflow correctly supports both unsafe blocking and safe approval scenarios.



This demonstrates that the clinical decision support workflow not only prevents unsafe medication combinations but also supports transactional prescription creation when no clinical risk is identified.

Lessons Learned

This project provided us with significant technical and architectural experience in designing and implementing a real-world healthcare management platform that uses multiple database technologies within a single integrated environment. Throughout the development process, several aspects of the project worked extremely well, while other areas presented technical and operational challenges that required adjustments and problem-solving from the team.

One of the most successful aspects of the project was the implementation of the polyglot database architecture itself. Separating workloads across PostgreSQL, MongoDB, and Neo4j significantly improved the overall system design and demonstrated the strengths of using specialized databases for different healthcare operations. PostgreSQL handled transactional healthcare records efficiently and reliably, MongoDB provided flexibility for managing semi-structured clinical notes, and Neo4j performed exceptionally well for modeling drug interaction relationships and clinical decision support workflows. The prescription safety engine was one of the strongest components of the system because it successfully demonstrated real-time drug interaction checks and contraindication analysis using graph traversal techniques.

Another area that went well was the normalized relational schema design. Proper entity relationships, foreign key constraints, and indexing strategies improved data integrity and query performance. The team's effective collaboration, database integration, and healthcare workflow modeling also contributed positively to the overall success of the project.

Despite the accomplishments, the team faced a number of challenges throughout the project phases. One major difficulty involved integrating multiple database technologies into a single coordinated system. Maintaining consistency between PostgreSQL, MongoDB, and Neo4j required careful design and validation logic. Synchronizing healthcare data across different databases occasionally introduces complexity during testing and debugging.

Another challenge involved schema evolution and data migration. As the project requirements evolved, updates to database schemas sometimes required modifications across multiple services. The team also experienced challenges with query optimization, particularly when handling cross-database workflows and larger healthcare datasets.

Team Contributions

Name	Task Completed/Role	Contribution Percentage
Pushkar Vishwas	Neo4j Lead	25
Etsubdink Workalemahu Yergashewa	Integration and DevOps Lead	25
Louis Tafah	Design and Documentation Lead	25
Adyasha Mishra	MongoDD Lead	25